

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине Построение и анализ алгоритмов
ТЕМА: «КРАТЧАЙШИЕ ПУТИ В ГРАФЕ: КОММИВОЯЖЁР»

Студент гр. 4343

Стрижков. И. А.

Преподаватель

Курасов Н. С.

Санкт-Петербург

2026

Цель работы

Изучить принцип работы метода ветвей и границ для точного решения задачи коммивояжёра (на основе алгоритма Литтла с модификацией через оценку минимального остовного дерева) и алгоритма ближайшего соседа (АБС) для приближённого решения, а также реализовать их на практике.

Задание.

Вариант 2

МВиГ: Алгоритм Литтла с модификацией: после приведения матрицы, к нижней оценке веса решения добавляется нижняя оценка суммарного веса остатка пути на основе МОД. Приближённый алгоритм: АБС. Замечание к варианту 2. Начинать АБС со стартовой вершины..

Независимо от варианта, при сдаче работы должна быть возможность генерировать матрицу весов (произвольную или симметричную), сохранять её в файл и использовать в качестве входных данных.

Алгоритм динамического программирования (итеративная реализация)

В волшебной стране Алгоритмии великий маг, Гамильтон, задумал невероятное путешествие, чтобы связать все города страны закланием процветания. Для этого ему необходимо посетить каждый город ровно один раз, создавая тропу благополучия, и вернуться обратно в столицу, используя минимум своих чародейских сил. Вашей задачей является помощь в прокладывании маршрута с помощью могущественного алгоритма динамического программирования.

Карта дорог Алгоритмии перед Гамильтоном представляет собой граф, где каждый город соединён магическими порталами с каждым другим. Стоимость использования портала из города в город занимает определённое количество маны, и Гамильтон стремится минимизировать общее потребление магической энергии для закрепления проклятия.

Входные данные:

Первая строка содержит одно целое число N (N - количество городов). Города нумеруются последовательными числами от 0 до $N-1$. Следующие N строк содержат по N чисел каждая, разделённых пробелами, формируя таким образом матрицу стоимостей M . Каждый элемент $M[i][j]$ этой

матрицы представляет собой затраты маны на перемещение из города i в город j . Если $M[i][j] = -1$, это означает, что прямого пути между городами нет.

Выходные данные:

Первая строка: Список из N целых чисел, разделённых пробелами, обозначающих оптимальный порядок городов в магическом маршруте Гамильтона. В начале идёт город 0, с которого начинается маршрут, затем последующие города до тех пор, пока все они не будут посещены, и в конце снова город 0.

Вторая строка: Число, указывающее на суммарное количество израсходованной маны для завершения пути.

Описание работы алгоритма:

Для точного решения задачи коммивояжёра используется метод ветвей и границ на основе алгоритма Литтла с модификацией через оценку минимального остовного дерева (МОД). Алгоритм последовательно разбивает множество всех туров на подмножества и для каждого подмножества вычисляет нижнюю оценку стоимости. Если нижняя оценка подмножества оказывается хуже уже найденного решения, подмножество отбрасывается без дальнейшего исследования, что существенно сокращает перебор по сравнению с полным перебором всех $(n-1)!$ вариантов.

Приближённый алгоритм: АБС (алгоритм ближайшего соседа)

Для больших матриц метод ветвей и границ может оказаться слишком затратным по времени. В таких случаях используется приближённый алгоритм - алгоритм ближайшего соседа (АБС), который быстро находит хороший, но не обязательно оптимальный маршрут.

Входные данные:

Первая строка содержит одно целое число N (N - количество городов).

Города нумеруются последовательными числами от 0 до $N-1$.

Следующие N строк содержат по N чисел каждая, разделённых пробелами, формируя таким образом матрицу стоимостей M . Каждый элемент $M[i][j]$ этой матрицы представляет собой затраты маны на перемещение из города i в город j .

Входные данные:

Программа выводит результаты работы двух алгоритмов - МВиГ (метод ветвей и границ, алгоритм Литтла с МОД-эвристикой) и АБС (алгоритм ближайшего соседа).

Первая строка: список из $N + 1$ целых чисел, разделённых символами “->”, обозначающих порядок городов в маршруте. Маршрут начинается со стартовой вершины, затем перечисляются все остальные города, и в конце снова указывается стартовая вершина (замыкание тура). Если тур не найден, выводится сообщение «Решение не найдено».

Вторая строка: одно целое число - суммарная стоимость найденного маршрута.

Описание работы алгоритма:

МВиГ (алгоритм Литтла с МОД-эвристикой)

1. Приводим матрицу стоимостей: из каждой строки вычитаем её минимум, затем из каждого столбца - его минимум. Сумма вычтенных значений - нижняя оценка стоимости тура.
2. Дополнительно строим минимальное остовное дерево (МОД) по ещё не пройденным вершинам - это вспомогательная эвристика для выбора ребра ветвления при равных штрафах.
3. Находим нулевой элемент приведённой матрицы с максимальным штрафом.
4. Ветвимся по выбранному ребру: включаем его в путь либо исключаем.

5. Для каждой ветви считаем нижнюю оценку и отбрасываем ветви, которые не могут улучшить текущее решение.
6. Когда выбрано $n-1$ рёбер, замыкаем тур и обновляем лучшее решение.
7. Повторяем шаги 3-6, пока очередь узлов не опустеет. В результате получаем точный оптимальный тур.

АБС (алгоритм ближайшего соседа)

1. Начинаем с тура, содержащего только стартовую вершину.
2. На каждом шаге из текущей вершины идём в ближайшую ещё не посещённую вершину.
3. Повторяем, пока все города не будут посещены.
4. Замыкаем тур возвратом в стартовую вершину.

Выполнение работы

Описание метода ветвей и границ (алгоритм Литтла с МОД-эвристикой)

Метод ветвей и границ - точный алгоритм решения задачи коммивояжёра, основанный на алгоритме Литтла с модификацией через оценку минимального остовного дерева (МОД). Он позволяет находить оптимальный маршрут по матрице стоимостей, последовательно отсекая подмножества туров, которые не могут улучшить уже найденное решение. Алгоритм состоит из следующих шагов:

1. Преобразование входной матрицы: отсутствующие рёбра (значение -1) заменяются на бесконечность, диагональ принудительно устанавливается в бесконечность (переход из вершины в саму себя запрещён).
2. Приведение матрицы: из каждой строки вычитается её минимальный элемент, затем из каждого столбца - его минимальный элемент. Сумма вычтенных значений называется константой приведения и является нижней оценкой стоимости любого тура в текущем подмножестве.
3. Дополнительно строится минимальное остовное дерево (МОД) по вершинам, ещё не вошедшим в частичный путь. Вес МОД используется как вспомогательная эвристика при выборе ребра ветвления: при равных штрафах предпочтение отдаётся ребру с большей МОД-оценкой остатка пути.
4. Выбор ребра для ветвления: находится нулевой элемент приведённой матрицы с максимальным штрафом (штраф = минимум в строке без данного столбца + минимум в столбце без данной строки).
5. Ветвление по выбранному ребру (i, j) :
 - ветвь 1: ребро включается в путь. В матрице запрещаются обратное ребро (j, i) , все рёбра из i и все рёбра в j . Матрица приводится заново, константа приведения добавляется к накопленной.

- ветвь 2: ребро исключается. Соответствующий элемент матрицы заменяется на бесконечность, матрица приводится заново.
6. Отсечение: для каждой ветви вычисляется нижняя оценка. Если она не меньше стоимости уже найденного тура, ветвь отбрасывается без дальнейшего исследования.
 7. Замыкание тура: когда выбрано $n-1$ рёбер, они образуют гамильтонов путь. Находится начало цепочки (вершина без входящего ребра) и конец цепочки (вершина без исходящего ребра). Тур замыкается ребром «конец - начало», и если суммарная стоимость меньше текущего лучшего решения, оно обновляется.
 8. Процесс повторяется, пока очередь узлов не опустеет. В результате находится точный оптимальный тур.

Описание АБС (алгоритм ближайшего соседа)

Алгоритм ближайшего соседа - приближённый (жадный) алгоритм решения задачи коммивояжёра. Он строит маршрут, последовательно переходя в ближайшую ещё не посещённую вершину. Алгоритм состоит из следующих шагов:

1. Начинаем с тура, содержащего только стартовую вершину.
2. На каждом шаге из текущей вершины выбирается ближайшая ещё не посещённая вершина, и осуществляется переход в неё.
3. Шаги повторяются, пока все города не будут посещены.
4. После включения всех городов тур замыкается возвратом в стартовую вершину.

Оценка сложности метода ветвей и границ:

Временная сложность:

- В худшем случае метод ветвей и границ исследует все возможные туры: $O(n!)$.

- На практике за счёт отсечения, по нижней оценке, количество исследуемых узлов существенно меньше.
- На каждом узле выполняется приведение матрицы - $O(n^2)$, и построение МОД - $O(n^2)$.
- Итоговая оценка: $O(n^2 \cdot 2^n)$ в худшем случае, но обычно значительно быстрее.

Пространственная сложность:

- Хранение матрицы для каждого узла дерева поиска: $O(n^2)$.
- Хранение пути и очереди узлов: $O(n \cdot K)$, где K - количество узлов в очереди.
- Итого: $O(n^2 \cdot K)$

Примечание: для $n \leq 10$ метод работает практически мгновенно. Для $n \approx 15-20$ время работы может увеличиваться, но за счёт МОД-эвристики и отсечения метод остаётся применимым для учебных задач. Для больших n целесообразно использовать приближённые методы.

Оценка сложности АБС

Временная сложность:

- На каждом из n шагов выполняется поиск ближайшей не посещенной вершины среди оставшихся (до n).
- Итого на шаг: $O(n)$.
- Общая сложность: $O(n^2)$.

Пространственная сложность:

- Хранение матрицы расстояний: $O(n^2)$
- Хранение текущего тура: $O(n)$
- Хранение массива посещённых вершин: $O(n)$
- Итого: $O(n^2)$

Примечание: Алгоритм работает быстро даже для больших n (1000+ городов), что делает его пригодным для практического использования.

Структура программы

Код программы содержит реализацию следующих функций:

Метод ветвей и границ (точный метод):

- `littleAlgorithm(costForLittle, costReal, startVertex)` - главная функция, реализующая алгоритм Литтла с МОД-эвристикой. Возвращает оптимальный путь и его стоимость.
- `reduceMatrix(m, n)` - приведение матрицы (вычитание минимумов по строкам и столбцам).
- `mstWeight(cost, verts)` - построение минимального остовного дерева (алгоритм Прима).
- `mstEstimate(cost, path, n, startVertex)` - МОД-оценка остатка пути по ещё не пройденным вершинам.
- `buildOrder(path, n, startVertex)` - восстановление порядка вершин тура из набора рёбер с поворотом цикла так, чтобы стартовая вершина была первой.

АБС (приближённый метод):

- `nearestNeighbor(cost, startVertex)` - главная функция, реализующая алгоритм ближайшего соседа. Строит приближённый маршрут, начиная со стартовой вершины.

Вспомогательные функции:

- `hasTour(cost)` - проверка существования тура (у каждой вершины должно быть хотя бы одно входящее и одно исходящее ребро).
- `printMatrix(m)` - вывод матрицы в читаемом виде (inf для отсутствующих рёбер).
- `generateRandomMatrix(cost, n, maxW, seed)` - генерация произвольной (несимметричной) матрицы.
- `generateSymmetricMatrix(cost, n, maxW, seed)` - генерация симметричной матрицы.
- `saveMatrixToFile(cost, startVertex, filename)` - сохранение матрицы и стартовой вершины в файл.

- `loadMatrixFromFile(cost, n, startVertex, filename)` - чтение матрицы и стартовой вершины из файла.
- `saveResultToFile(path, total, method, filename)` - сохранение результатов работы алгоритма в файл.
- `askN()`, `askStart(n)`, `askFilename(prompt, def)` - вспомогательные функции для корректного ввода данных с клавиатуры.

Тестирование

Программа была протестирована на различных входных данных. Для удобства тестирования, создан генератор матриц, подходящий для обоих алгоритмов. Рёбра на главной диагонали – -1 (отсутствие пути в саму себя), а остальные - целые или вещественные числа. Отсутствующие рёбра между разными городами также обозначаются -1.

Таблица 1.

Входные данные	Выходные данные	Комментарий
3 -1 1 3 3 -1 1 1 2 -1	0 1 2 0 3.0	Алгоритм МВиГ
4 -1 3 4 1 1 -1 3 4 9 2 -1 4 8 9 2 -1	0 3 2 1 6.0	Алгоритм МВиГ
3 -1 1 1 1 -1 1 1 1 -1	0 1 2 0 3.0	Алгоритм МВиГ
3 -1 1 3 3 -1 1 1 2 -1	0 1 2 0 3.0	Алгоритм АБС
4 -1 3 4 1 1 -1 3 4	0 3 2 1 6.0	Алгоритм АБС

9 2 -1 4 8 9 2 -1		
3 -1 1 1 1 -1 1 1 1 -1	0 1 2 0 3.0	Алгоритм АБС

```

Выберите способ ввода:
1 – ввод с клавиатуры
2 – чтение из файла
3 – генерация произвольной (несимметричной) матрицы
4 – генерация симметричной матрицы
Ваш выбор: 1
Введите количество вершин (>= 2): 3
Введите матрицу весов (3x3).
Диагональ можно вводить как 0 или -1. Для отсутствия ребра: -1 или большое число.

-1
1
3
3
-1
1
1
2
-1
Введите стартовую вершину (0..2): 0

=== Исходная матрица ===
inf    1    3
 3   inf    1
 1    2   inf

Сохранить матрицу в файл? (y/n): n

=== МВиГ (Литтл + МОД-эвристика) ===
Путь: 0 -> 1 -> 2 -> 0
Стоимость: 3

=== Приближённый алгоритм АБС (ближайший сосед) ===
Путь: 0 -> 1 -> 2 -> 0
Стоимость: 3

```

Рисунок 1 – Результат работы программы

Оптимальный маршрут (МВиГ) — стоимость: 90

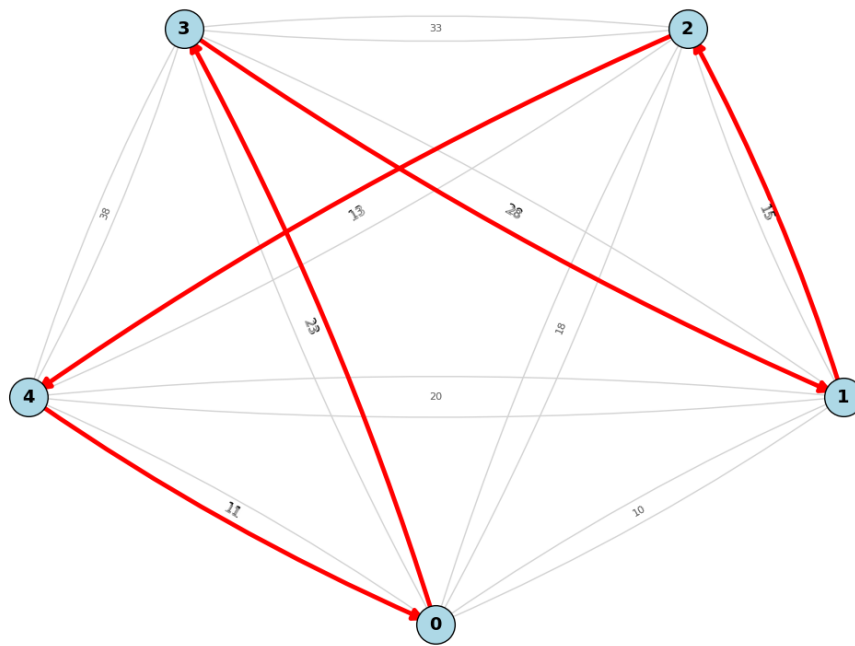


Рисунок 2 – Визуализация МВиГ

Приближённый маршрут (АБС) — стоимость: 99

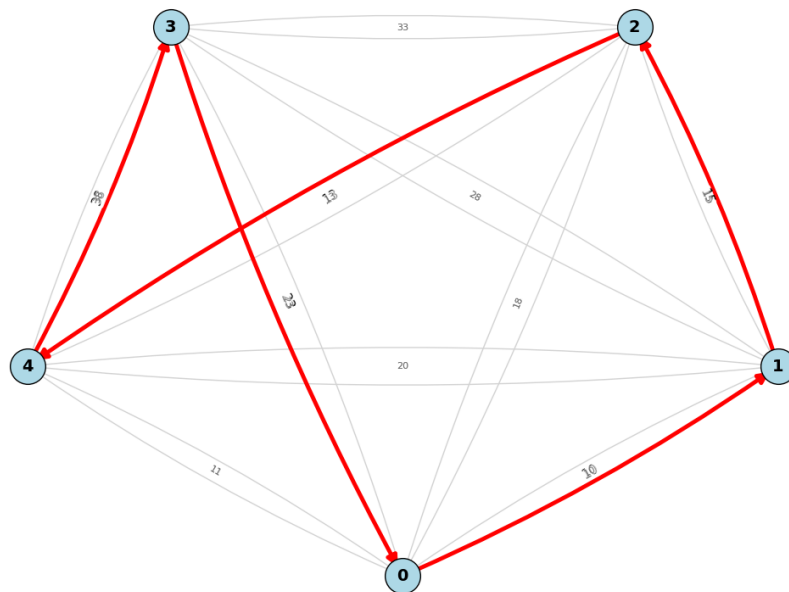


Рисунок 3 – Визуализация АБС

Исследование

Исследуем эффективность обоих алгоритмов на входных данных разного размера.

Таблица 2. Исследование эффективности по времени.

МВиГ (точный метод)	
N	Время в секундах
5	0.000055
7	0.000118
9	0.000268
11	0.000463
13	0.001061
АСБ (приближённый метод)	
N	Время в секундах
100	0.000257
300	0.002173
500	0.005800
700	0.011133
900	0.018853

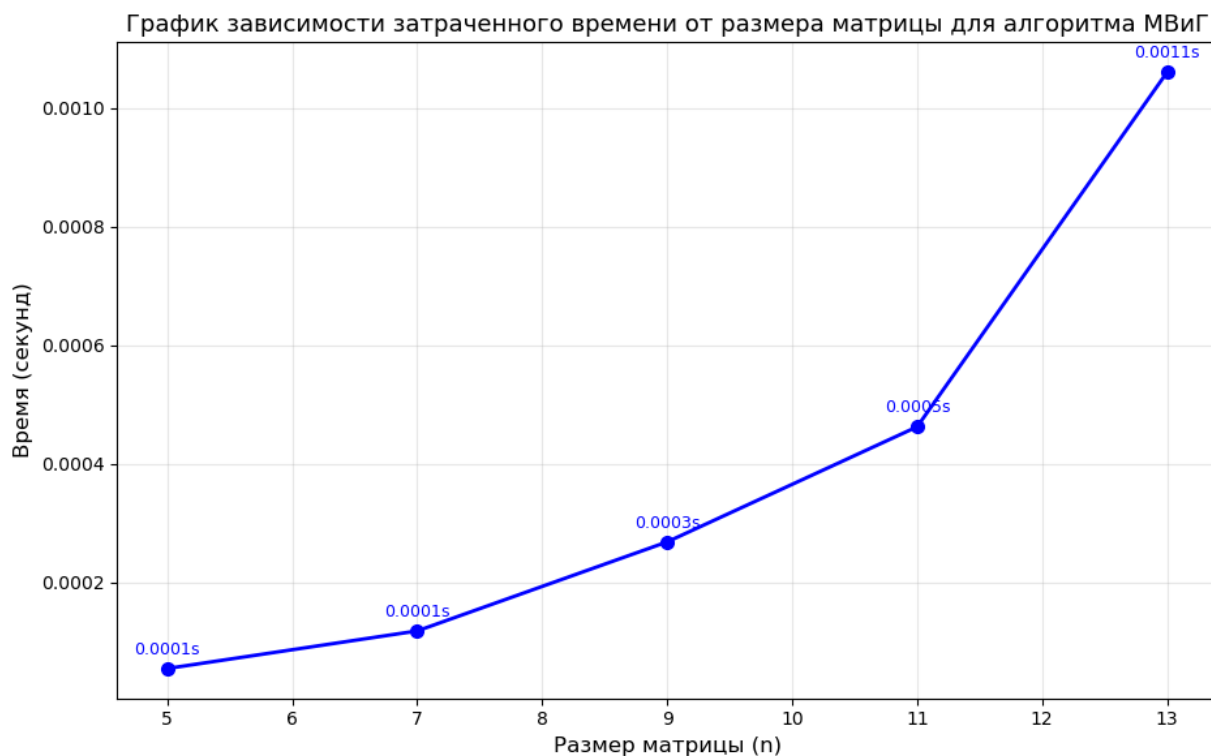


Рисунок 3 – График зависимости затраченного времени от размера матрицы для алгоритма МВиГ.



Рисунок 4 – График зависимости затраченного времени от размера матрицы для алгоритма АБС.

Анализ полученных данных

Алгоритм динамического программирования (DP):

- Наблюдается нелинейный (экспоненциальный) рост времени выполнения с увеличением размера матрицы.
- При увеличении N с 5 до 13 время возрастает примерно в 15-20 раз
- Это объясняется теоретической сложностью метода ветвей и границ: в худшем случае $O(n!)$, на практике - $O(n^2 \cdot 2^n)$ с учётом отсечения.
- Алгоритм применим для $n \leq 15-16$, при больших значениях время становится неприемлемым

Алгоритм АБС:

- Наблюдается квадратичный рост времени выполнения ($O(n^2)$)
- При увеличении N с 100 до 9000 время возрастает примерно в 70-75 раз
- При $N=900$ время составляет около 0.019 секунды, что приемлемо для практического использования
- Алгоритм может работать с тысячами городов его пригодным для больших практических задач.

Выводы

Во время выполнения лабораторной работы были изучены принципы работы метода ветвей и границ (алгоритм Литтла с модификацией через оценку минимального остовного дерева) для точного решения задачи коммивояжёра и алгоритма ближайшего соседа (АБС) для приближённого решения. Были решены задачи поиска точного и приближённого маршрута коммивояжёра.